Plaza Works Job Management App

Architecture Guidelines: Security, Performance and Reliability

---

Purpose of This Document

These are the rules and standards for building your app. Share this with any developer or AI assistant helping you build. It protects your business, your clients, and your team's data.

Part 1: Security Rules

1.1 Authentication (Who Can Log In)

Rules

- Every user must have a unique login (no shared accounts)
- Passwords must be minimum 10 characters with mixed case and numbers
- Use an established auth provider (Clerk, Supabase Auth, or Auth0) - never build your own
- Session tokens expire after 24 hours of inactivity
- Failed login attempts: lock account after 5 failures for 15 minutes

Client Portal

- Separate authentication system from internal staff
- Clients can only see their own jobs - never other clients' data
- Magic link login option (more secure than passwords for clients)

1.2 Authorization (Who Can Do What)

Rules

- Every action checks permissions before executing
- Permissions checked on the server, not just the interface

- Default to no access - only grant what's explicitly needed
- Log all permission changes with who made them and when

Permission Levels

View Only → Can see data but not change it
Edit → Can modify existing records
Create → Can add new records
Delete → Can remove records (rare - most things should be archived instead)
Admin → Can change permissions and system settings

Data Isolation

- Staff see only jobs they're assigned to (unless manager/admin)
- Contractors never see financial data or other contractors' details
- Clients see only their own information

1.3 Data Protection

Sensitive Data Categories

Category	Examples	Protection Level
Critical	Passwords, payment details, API keys	Encrypted, never logged, never displayed
High	Client addresses, phone numbers, emails	Encrypted at rest, access logged
Medium	Job details, schedules, quotes	Access controlled by permissions
Low	General settings, templates	Standard protection

Encryption Rules

- All data encrypted at rest (database encryption enabled)
- All data encrypted in transit (HTTPS only, no HTTP)
- Payment card details never stored - use Stripe/payment provider tokens
- API keys and secrets stored in environment variables only
- Database backups encrypted

Data Handling

- Never log sensitive data (passwords, card numbers, personal details)
- Mask sensitive fields in the interface (show last 4 digits only)

- Auto-delete old session data after 90 days
- Client data retained for 7 years (legal requirement), then archived/deleted

---

1.4 Input Validation

Rules

- Validate all user input on the server (never trust the browser)
- Reject unexpected data formats
- Sanitize all text inputs to prevent code injection
- File uploads: check file type, limit size to 10MB, scan for malware
- Phone numbers: validate format before saving
- Email addresses: validate format and consider verification

Photo Uploads

- Accept only: JPG, PNG, HEIC, WebP
- Strip metadata (EXIF) to remove GPS coordinates from images
- Resize to maximum 2000px width (saves storage, improves speed)
- Store in separate file storage (Cloudinary or S3), not the database

---

1.5 API Security

Rules

- All API endpoints require authentication
- Rate limiting: maximum 100 requests per minute per user
- Webhook endpoints use signature verification
- CORS configured to allow only your domains
- No sensitive data in URLs (use POST body or headers)

Third-Party Integrations

- Use official SDKs where available
- Store API keys in environment variables
- Rotate API keys every 12 months
- Monitor for unusual API usage

1.6 Audit Logging

What to Log

- All logins (successful and failed)
- Permission changes
- Data exports
- Quote and invoice creation
- Payment events
- Job status changes
- User account changes

Log Format

Timestamp | User ID | Action | Target | IP Address | Success/Failure

Log Retention

- Security logs: 2 years minimum
- Activity logs: 1 year
- Never log passwords or sensitive data

Part 2: Performance Rules

2.1 Database Design

Rules

- Every table has a primary key
- Add indexes to fields you search or filter by frequently
- Use foreign keys to link related data
- Archive old data instead of keeping everything in active tables
- Avoid storing large files in the database - use file storage

Common Indexes Needed

- Jobs: status, client_id, assigned_user_id, created_date
- Users: email, role
- Invoices: job_id, status, due_date
- Schedule: date, user_id

Data Archiving

- Jobs completed over 2 years ago: move to archive table
- Keep counts and summaries in main tables for reporting

- Archived data still searchable but stored separately

2.2 Query Efficiency

Rules

- Never fetch all records - always paginate (50 items per page default)
- Only select the fields you need, not entire records
- Avoid queries inside loops
- Use database joins instead of multiple separate queries
- Cache frequently accessed data that doesn't change often

Examples

Bad: Fetch all 5000 jobs, then filter in code
Good: Fetch only jobs matching the filter, limit 50

Bad: For each job, fetch the client separately
Good: Join jobs and clients in one query

2.3 Caching

What to Cache

- User permissions (refresh on change)
- Dropdown options (roles, job types, statuses)
- Dashboard statistics (refresh every 5 minutes)
- Client portal job status (refresh every 2 minutes)

What Not to Cache

- Financial data (always fetch fresh)
- Real-time location data
- Anything that must be immediately accurate

2.4 File Handling

Rules

- Compress images before storing

- Use a CDN for serving images and files
- Generate thumbnails for photo galleries
- Lazy load images (only load when visible on screen)
- Set proper cache headers on static files

Photo Storage Structure

/jobs/{job_id}/photos/{date}_{timestamp}_{original_filename}

2.5 Frontend Performance

Rules

- Minimize JavaScript bundle size
- Load only what's needed for each page
- Show loading states - never leave users guessing
- Prefetch likely next pages
- Work offline where possible (view schedules, draft reports)

Mobile Considerations

- Test on slow 3G connections
- Compress all assets
- Minimize data transfer for field workers
- Support poor connectivity (queue actions when offline)

Part 3: Reliability Rules

3.1 Error Handling

Rules

- Every action has success and failure handling
- Show clear error messages (not technical jargon)
- Log all errors with context
- Critical errors trigger alerts
- Never show stack traces to users

User-Friendly Errors

Bad: "Error 500: NullPointerException at line 234"
Good: "Something went wrong saving your changes. Please try again or contact support."

Error Severity Levels

Level	Response	Example
Critical	Immediate alert to admin	Payment processing down
High	Alert within 1 hour	Login failures spiking
Medium	Daily review	Slow page loads
Low	Weekly review	Minor UI glitches

3.2 Data Backup

Rules

- Automated daily backups
- Backups stored in different location from main database
- Weekly backup verification (test that restore works)
- Keep 30 days of daily backups
- Keep 12 months of monthly backups
- Document restore procedure

Recovery Targets

- Recovery Point Objective (RPO): Maximum 24 hours data loss
- Recovery Time Objective (RTO): Back online within 4 hours

3.3 Uptime and Monitoring

Monitoring Required

- Server/app health check every 5 minutes
- Database connection monitoring
- API endpoint response times
- Error rate tracking
- Storage space alerts

Alerting

- Downtime: Immediate SMS/email alert

- High error rate: Alert within 15 minutes
- Slow performance: Alert within 1 hour

Uptime Target

- 99.5% uptime (allows about 1.8 days downtime per year)
- Scheduled maintenance windows: notify users 48 hours ahead

3.4 Graceful Degradation

Rules

- If a feature fails, the rest of the app should keep working
- If external service is down, show clear message and queue actions
- Critical functions have fallback options

Examples

SMS service down → Queue messages, notify admin, retry hourly
Photo upload fails → Save job without photo, prompt to retry later
Payment check fails → Allow job creation, flag for manual verification

---

3.5 Testing Requirements

Before Any Release

- Test all user roles can log in
- Test a complete job workflow (create → schedule → complete)
- Test client portal access
- Test on mobile devices
- Verify backups are current

Automated Tests (Ideal)

- Authentication flows
- Permission checks
- Critical calculations (quotes, productivity metrics)
- Payment link generation

Part 4: Development Rules

4.1 Code Organization

Structure

```
/app
  /api          → Backend endpoints
  /components   → Reusable interface elements
  /pages        → Main screens
  /utils        → Helper functions
  /hooks        → Shared logic
  /types        → Data definitions
/database
  /migrations   → Database changes
  /seeds        → Test data
/docs           → Documentation
```

Naming Conventions

- Files: lowercase with hyphens (job-list.tsx)
- Components: PascalCase (JobList)
- Functions: camelCase (getJobById)
- Database tables: lowercase with underscores (job_items)
- Constants: UPPERCASE (MAX_FILE_SIZE)

4.2 Environment Management

Environments Needed

Environment	Purpose	Data
Development	Building and testing	Fake test data only
Staging	Final testing before release	Copy of real structure, fake data
Production	Live system	Real data

Environment Variables

- Never commit secrets to code
- Use Replit Secrets for API keys

- Different keys for each environment
- Document all required variables

Required Variables

DATABASE_URL
AUTH_SECRET
STRIPE_API_KEY
TWILIO_ACCOUNT_SID
TWILIO_AUTH_TOKEN
SMS_FROM_NUMBER
EMAIL_API_KEY
FILE_STORAGE_KEY

4.3 Version Control

Rules

- All code in Git/GitHub
- Never commit directly to main branch
- Use descriptive commit messages
- Review changes before deploying
- Tag releases with version numbers

Commit Message Format

[Module] Brief description of change

Examples:
[Auth] Add password reset flow
[Jobs] Fix scheduling conflict detection
[Client Portal] Add PC items approval workflow

4.4 Documentation

Required Documentation

- Setup instructions for new developers
- Environment variables list with descriptions
- API endpoint documentation
- Database schema diagram
- User guides per role

Code Comments

- Explain why, not what
- Document complex business logic
- Note any workarounds and why they exist

Part 5: Compliance Checklist

Before Launch

Security

- [ ] All passwords hashed (never stored plain text)
- [ ] HTTPS enforced everywhere
- [ ] Sensitive data encrypted
- [ ] Permissions tested for all roles
- [ ] API rate limiting enabled
- [ ] Input validation on all forms
- [ ] File upload restrictions working
- [ ] Audit logging active

Performance

- [ ] Database indexes created
- [ ] Pagination working on all lists
- [ ] Images compressed and served via CDN
- [ ] Load tested with expected user count

Reliability

- [ ] Automated backups running
- [ ] Backup restore tested
- [ ] Monitoring alerts configured
- [ ] Error handling reviewed
- [ ] Downtime notification process defined

Privacy

- [ ] Privacy policy published
- [ ] Terms of service published
- [ ] Data retention policy documented
- [ ] Client data export capability
- [ ] Account deletion process

Part 6: Extensibility and Integration Rules

Why This Matters

You want to add new features later without rebuilding everything. You also want to connect to other tools (accounting software, CRMs, mapping services). This section ensures your app is built to grow and play nicely with others.

6.1 Modular Architecture

Core Principle

Each module should work independently. If you remove or change one module, the others keep working.

Rules

- Each module has its own folder, database tables, and API endpoints
- Modules communicate through defined interfaces, not by reaching into each other's code
- No module directly accesses another module's database tables
- Shared functionality lives in a common utilities folder

Module Independence Test

Ask: "Could I replace this module with a completely different implementation without changing other modules?"

If yes, your architecture is correct.

Module Structure

```
/modules
  /users
    /api          → User-related endpoints
    /components   → User-related interface elements
    /services     → User business logic
    /types        → User data definitions
    index.ts      → Public exports only
```

```
/jobs
  /api
  /components
  /services
  /types
  index.ts
/scheduling
  ...
/invoicing
  ...
```

Public vs Private

- Each module exposes only what other modules need (public interface)
- Internal workings stay hidden (private)
- Changes to private code should never break other modules

6.2 API-First Design

Core Principle

Every feature is built as an API first, interface second. This means any feature can be accessed by other systems, not just your own interface.

Rules

- Build the API endpoint before building the interface
- Every action available in the interface must also be available via API
- APIs return consistent, predictable data formats
- Document every endpoint as you build it

API Structure

```
/api/v1/jobs            → List jobs, create job
/api/v1/jobs/{id}       → Get, update, delete specific job
/api/v1/jobs/{id}/photos  → Job photos
/api/v1/jobs/{id}/tasks   → Job tasks

/api/v1/clients         → List clients, create client
/api/v1/clients/{id}    → Get, update specific client
/api/v1/clients/{id}/jobs → Jobs for this client

/api/v1/users           → Staff management
/api/v1/schedule        → Scheduling
/api/v1/invoices        → Invoicing
```

/api/v1/vehicles → Fleet management

Versioning

- Always include version in API path (/api/v1/, /api/v2/)
- When you make breaking changes, create a new version
- Keep old versions running until all consumers have migrated
- Minimum 6 months notice before retiring an API version

Standard Response Format

```
{
  "success": true,
  "data": { ... },
  "meta": {
    "page": 1,
    "per_page": 50,
    "total": 234
  }
}

{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid email format",
    "field": "email"
  }
}
```

6.3 Event-Driven Communication

Core Principle

Instead of modules calling each other directly, they announce events. Other modules listen and react. This keeps things loosely connected.

How It Works

Job Module: "Hey, a job was completed!" (publishes event)

Invoicing Module: (listening) "I'll generate the final invoice"
Notifications Module: (listening) "I'll tell the client"
Analytics Module: (listening) "I'll update the stats"

Benefits

- Adding new reactions doesn't require changing the original module
- Modules don't need to know who's listening
- Easier to add new features later

Core Events to Implement

Event	Triggered When	Likely Listeners
job.created	New job added	Scheduling, Notifications
job.status_changed	Job moves to new stage	Notifications, Client Portal, Analytics
job.completed	Job finished	Invoicing, Analytics, Notifications
payment.received	Payment confirmed	Jobs, Notifications, Analytics
schedule.updated	Schedule changes	Notifications, Client Portal
user.assigned	Worker assigned to job	Notifications, Scheduling
client.action_required	Client needs to do something	Notifications
quote.approved	Client accepts quote	Jobs, Invoicing, Scheduling
pc_items.selected	Client picks PC items	Jobs, Notifications
vehicle.service_due	Vehicle needs servicing	Notifications
task.completed	Worker finishes task	Jobs, Analytics
photo.uploaded	New photo added	Jobs, Client Portal

Event Format

```
{
  "event": "job.completed",
  "timestamp": "2024-01-15T14:30:00Z",
  "data": {
    "job_id": "job_123",
    "client_id": "client_456",
    "completed_by": "user_789"
  },
  "source": "jobs_module"
}
```

Implementation Options

- Simple: Database table as event queue
- Medium: Redis pub/sub
- Advanced: Dedicated message queue (RabbitMQ, AWS SQS)

Start simple. Move to more advanced options if you need better performance or reliability.

6.4 Database Design for Growth

Core Principle

Design your database so adding new features doesn't require rebuilding existing tables.

Rules

- Each module owns its tables (jobs module owns jobs table)
- Link between modules using IDs only
- Use a settings/metadata table for flexible configuration
- Plan for custom fields from the start

Flexible Custom Fields

Instead of adding columns for every new field, use a metadata approach:

job_custom_fields table:
- job_id
- field_name
- field_value
- field_type

Example rows:
job_123 | "warranty_period" | "24" | "number"
job_123 | "special_instructions" | "Access via back gate" | "text"

Lookup Tables for Extensibility

Don't hardcode statuses, types, or categories. Use lookup tables:

job_statuses table:
- id
- name
- display_order
- is_active
- triggers_notification

This way you can add new statuses without code changes.

Soft Deletes

Never actually delete records. Mark them as deleted:

- is_deleted: true/false
- deleted_at: timestamp
- deleted_by: user_id

This protects data integrity and allows recovery.

Audit Trail Table

Track all changes to important records:

audit_log table:
- id
- table_name
- record_id
- action (create/update/delete)
- old_values (JSON)
- new_values (JSON)
- changed_by
- changed_at

6.5 Third-Party Integration Framework

Core Principle

Build a consistent way to connect external apps. When you add a new integration, it should follow the same pattern as existing ones.

Integration Types

Type	Description	Examples
Outbound Push	Your app sends data out	Send invoice to Xero, SMS via Twilio
Inbound Push	External app sends data to you	Payment webhook from Stripe
Outbound Pull	Your app fetches data	Get calendar from Google

Inbound Pull	External app fetches from you	Another system reads your API

Standard Integration Structure

```
/integrations
  /xero
    config.ts       → Connection settings
    auth.ts         → Authentication handling
    sync.ts         → Data sync logic
    webhooks.ts     → Incoming webhook handlers
    types.ts        → Xero-specific data types
  /twilio
    ...
  /stripe
    ...
  /google-maps
    ...
```

Integration Config Table

```
integrations table:
- id
- name (xero, twilio, stripe)
- is_enabled
- credentials (encrypted JSON)
- settings (JSON)
- last_sync_at
- sync_status
```

Webhook Handler Pattern

All incoming webhooks should:

1. Verify the source (signature validation)
2. Log the raw payload
3. Parse and validate the data
4. Trigger appropriate internal events
5. Return acknowledgment quickly
6. Process heavy work in background

```
Incoming Stripe webhook:
1. Verify Stripe signature
2. Log: "Received payment.succeeded webhook"
3. Parse payment details
4. Trigger: payment.received event
5. Return 200 OK
```

6. Background: Update invoice, notify client

Integration Health Monitoring

Track for each integration:

- Last successful sync
- Error count in last 24 hours
- Average response time
- Current status (healthy/degraded/down)

Alert if any integration shows problems.

6.6 Common Integration Patterns

Accounting Software (Xero, MYOB, QuickBooks)

Sync Strategy:

- Clients: Two-way sync (create in either system)
- Invoices: Push from your app to accounting
- Payments: Pull from accounting to your app
- Products/Services: Pull from accounting as templates

Data Mapping:

```
Your App        →  Accounting Software
client          →  contact
invoice         →  invoice
job             →  (no equivalent - your app only)
payment         ←  payment
```

Calendar Systems (Google Calendar, Outlook)

Sync Strategy:

- Push scheduled jobs to external calendar
- Pull availability/conflicts from external calendar
- Two-way for staff calendars

Data Mapping:

```
Your App        →  Calendar
scheduled_job   →  event
```

job.address → event.location
assigned_users → event.attendees

Communication (Twilio, SendGrid)

Pattern:

- Create internal notification record first
- Send via external service
- Update record with delivery status
- Handle delivery webhooks for confirmations

Payment Processing (Stripe)

Pattern:

- Create payment intent via API
- Generate payment link
- Store intent ID locally
- Wait for webhook confirmation
- Never assume payment succeeded without webhook

Mapping Services (Google Maps)

Use Cases:

- Geocode addresses when saving client/job
- Calculate travel times for scheduling
- Display job locations on map
- Optimize route for multiple jobs

Caching:

- Cache geocoding results (addresses rarely change)
- Cache travel times for common routes
- Refresh travel times if older than 1 week

6.7 Building New Integrations Checklist

When adding any new integration, follow this checklist:

Planning

- [ ] What data flows in which direction?
- [ ] What's the sync frequency needed?

- [ ] What happens if the integration is down?
- [ ] What permissions/scopes are needed?

Security

- [ ] API keys stored in environment variables
- [ ] Webhook signatures verified
- [ ] Sensitive data not logged
- [ ] Credentials refreshed/rotated as required

Implementation

- [ ] Follows standard integration folder structure
- [ ] Config stored in integration table
- [ ] Error handling with retries
- [ ] Health monitoring in place

Testing

- [ ] Test with sandbox/test account first
- [ ] Verify webhook handling
- [ ] Test failure scenarios
- [ ] Load test if high volume expected

Documentation

- [ ] Setup instructions documented
- [ ] Required credentials/scopes listed
- [ ] Data mapping documented
- [ ] Troubleshooting guide

6.8 Future-Proofing Tips

Design for Features You Don't Have Yet

Multi-Location Support Even if you have one location now, add location_id to relevant tables:

jobs.location_id
users.location_id
vehicles.location_id

Multi-Currency Store currency code with all monetary values:

quotes.total_amount

quotes.currency (AUD, NZD, USD)

Time Zones Store all times in UTC, convert for display:

```
scheduled_at: 2024-01-15T03:00:00Z (UTC)
Display: "Monday 15 Jan, 2:00 PM" (Sydney time)
```

Translations Structure text so it can be translated later:

```
Instead of: "Job completed successfully"
Use: translate('job.completed.success')
```

Plugin Architecture (Advanced)

If you want others to extend your app later:

```
/plugins
  /custom-reports
    manifest.json    → Plugin metadata
    index.ts         → Entry point
    permissions.ts   → Required permissions
```

Plugin manifest example:

```
{
  "name": "Custom Reports",
  "version": "1.0.0",
  "author": "Third Party Dev",
  "hooks": ["job.completed", "payment.received"],
  "permissions": ["read:jobs", "read:payments"],
  "settings_ui": true
}
```

This is advanced - only consider if you plan to allow third-party extensions.

6.9 Integration Priority Recommendations

Must Have (Phase 1-2)

Integration	Purpose

Stripe	Payment links and processing
Twilio	SMS notifications
SendGrid/Resend	Email notifications
Google Maps	Address validation, travel times

Should Have (Phase 3-4)

Integration	Purpose
Xero/MYOB	Accounting sync
Google Calendar	Staff scheduling
Cloudinary/S3	Photo storage

Nice to Have (Phase 5+)

Integration	Purpose
Vehicle GPS	Real-time tracking
Zapier	Connect to anything
Slack	Internal notifications
Power BI	Advanced reporting

6.10 API for External Access

If you want other systems to pull data from your app:

Public API Endpoints (Authenticated)

```
GET  /api/v1/jobs              → List jobs
GET  /api/v1/jobs/{id}         → Single job details
GET  /api/v1/schedule          → Schedule data
GET  /api/v1/clients/{id}      → Client details
POST /api/v1/webhooks/subscribe  → Register for events
```

API Key Management

- Generate API keys per integration
- Set permissions per key (read-only, specific modules)
- Track usage per key
- Ability to revoke keys instantly

Rate Limiting for External APIs

- Standard: 100 requests per minute
- Burst: Allow 200 for 10 seconds
- Daily cap: 10,000 requests
- Higher limits available by request

Documentation

- Auto-generate API docs (OpenAPI/Swagger)
- Include example requests and responses
- Keep docs in sync with actual API

Quick Reference Card

The Non-Negotiables

1. **Never store passwords in plain text** - always hashed
2. **Never store payment card details** - use payment provider tokens
3. **Never trust user input** - validate everything server-side
4. **Never commit secrets to code** - use environment variables
5. **Never skip backups** - automate and verify them
6. **Never show technical errors to users** - log them, show friendly messages
7. **Never give more access than needed** - default to restricted
8. **Always use HTTPS** - no exceptions
9. **Always log security events** - logins, permission changes, data access
10. **Always test before deploying** - especially auth and payments

---

Questions to Ask Any Developer

Before hiring help or accepting AI-generated code, ask:

1. How are you handling authentication?
2. Where are API keys stored?
3. How is sensitive data encrypted?
4. What happens when something fails?

5. How are database queries optimized?
6. What monitoring is in place?
7. How are backups handled?

If they can't answer clearly, that's a red flag.